MODULE 20 · DOMAIN 4 - PERFORMANCE OPTIMIZATION, QUERYING, AND TRANSFORMATION

Clustering, materialized views, search optimization, QAS

Four features that all make queries faster. The whole game is knowing which one fits.

DOMAIN 4 of 5 WEIGHT 21% MODULE 20 of 31

admin@sthithapragna.com

They are not competitors. Each one fixes a **different shape of query**, and two of them can be used together.

The four at a glance

FEATURE	WHAT IT ACTUALLY DOES	EDITION
Clustering key	Reorganises the physical layout so related rows share partitions	Standard
Materialized view	Precomputes and maintains a result over one table	Enterprise
Search optimization	Builds a search access path beside the table for point lookups	Enterprise
Query acceleration	Temporarily leases extra serverless compute for scan and filter work	Enterprise

ON THE EXAM

Only clustering is Standard Edition. If a scenario names Standard, three of these four are already ruled out.

Clustering and materialized views, briefly

- **Clustering** suits **range scans** and equality on **medium-cardinality** columns. Large tables only, and not write-heavy ones.
- Ordered **lowest cardinality first**, three or four columns at most. A unique identifier is the classic bad key.
- **Materialized views** suit a **repeated aggregation** that must always be current, with **no lag**.
- **One table, no joins, no window functions.** That restriction rules them out more often than anything else.

> ON THE EXAM
>
> Modules 3 and 4 in four lines. If either restriction bites, the answer is one of the other two features.

Search Optimization: what it accelerates

- **Selective point lookups** returning one or a few rows from a very large table. High-cardinality columns.
- **Substring and regular expression** searches: LIKE, ILIKE, RLIKE.
- Predicates on **VARIANT, OBJECT and ARRAY**: equality, IN, ARRAY_CONTAINS, ARRAYS_OVERLAP, full-text SEARCH, and NULL checks.
- Selected **geospatial functions** on GEOGRAPHY values.
- Individually supported predicates joined by **AND** remain supported.

> ON THE EXAM
>
> The natural sentence is 'find me the one row with this exact value'. That is a needle in a haystack, not a range scan.

Search Optimization: enabling and cost

- **ALTER TABLE … ADD SEARCH OPTIMIZATION**, optionally **ON EQUALITY(col)**, **ON SUBSTRING(col)** or **ON GEO(col)**.
- **Enabling at table level does NOT enable it for semi-structured columns.** VARIANT, OBJECT and ARRAY need an explicit ON clause.
- Costs **storage** for the access path, typically about **a quarter of the table's size**, plus **serverless compute** for maintenance.
- Estimate first with **SYSTEM$ESTIMATE_SEARCH_OPTIMIZATION_COSTS**. Building the path can take significant time on a large table.

> ON THE EXAM
>
> That second point is a favourite. A team enables it table-wide, sees no improvement on their JSON queries, and cannot work out why.

Query Acceleration: borrowed compute

- Leases **extra serverless compute temporarily** for one query, then releases it. Your warehouse stays the size it was.
- Enabled per warehouse: **ENABLE_QUERY_ACCELERATION = TRUE**, with **QUERY_ACCELERATION_MAX_SCALE_FACTOR** capping how much it may lease.
- It accelerates the **scan and filter** portions only. **Aggregation and sorting are not accelerated.**
- The shape it suits: **large scans with an aggregation or a selective filter**, especially unpredictable ad hoc ones.

> ON THE EXAM
>
> This is the answer when occasional huge queries would otherwise force you to oversize a warehouse permanently.

Query Acceleration: eligibility and monitoring

- **Ineligible** when: filters are not selective enough, GROUP BY cardinality is too high, certain LIMIT clauses, or non-deterministic functions such as **SEQ or RANDOM**.
- **SYSTEM$ESTIMATE_QUERY_ACCELERATION(query_id)** returns eligible or ineligible, plus estimated times per scale factor.
- **ACCOUNT_USAGE.QUERY_ACCELERATION_ELIGIBLE** lists eligible queries. Latency up to **180 minutes**.
- That view **only shows queries not yet accelerated**. Once QAS is on and working, they disappear from it.
- Now **enabled by default** in Gen2 standard warehouses and multi-cluster warehouses.

> ON THE EXAM
>
> The 'only shows not-yet-accelerated queries' detail explains an empty result that looks like a problem and is actually success.

Snowflake Optima: the free one

- **Standard Edition, no additional cost**, but requires a **Gen2 or Adaptive warehouse**.
- **Optima Indexing** creates hidden indexes for recurring point lookups. **Optima Metadata** improves pruning. **Optima Planning** improves plans.
- **Best-effort.** Search Optimization costs credits and gives **consistent** index freshness for workloads that need guaranteed performance.
- Too new to be scored heavily. **Answer Search Optimization** when a question asks for guaranteed point-lookup performance.

> ON THE EXAM
>
> Recorded here because it looks like a free alternative to Search Optimization. It is not a replacement, and it is not on the exam yet.

The tiebreaker table

THE SYMPTOM	THE FEATURE	WHY
Range scan, poor pruning, medium-cardinality filter	Clustering key	Moves the data so pruning works
Point lookup, high-cardinality column, huge table	Search optimization	An access path for needles
Substring, regex, VARIANT or geospatial	Search optimization	The only one supporting these
Repeated aggregation, one table, always current	Materialized view	Precomputed, no lag
Repeated aggregation across a join, lag acceptable	Dynamic table	Materialized views cannot join
Occasional huge ad hoc scans	Query acceleration	Borrows compute for one query
The account is on Standard Edition	Clustering key	The other three need Enterprise

ON THE EXAM

Seven rows. If you memorise one table from this domain, memorise this one.

> A support application looks up individual customer records by a UUID from a 40 TB table, one row at a time, thousands of times an hour. Query Profile shows partitions scanned close to partitions total on every lookup.
>
> An engineer proposes a clustering key on the UUID column. The account is on Enterprise Edition.

What should be done?

A Proceed with the clustering key on the UUID. Poor pruning always means clustering.

B Enable search optimization on the UUID column. Point lookups on a unique column are exactly what it is for.

C Create a materialized view keyed on the UUID so lookups read the view instead.

D Enable query acceleration on the warehouse, which will accelerate the scans.

◆ PAUSE AND DECIDE

What should be done?

A. Clustering key on the UUID.

A unique identifier is the classic bad clustering key. ruinous reclustering cost, and it prunes nothing useful for a range.

B. Search optimization on the UUID column.

Selective point lookups on a high-cardinality column are its headline use. It builds an access path rather than moving data.

C. A materialized view keyed on the UUID.

A materialized view precomputes an aggregate. It does not act as an index for arbitrary single-row lookups.

D. Query acceleration.

QAS speeds up large scans. These are point lookups, and the fix is to avoid scanning rather than to scan faster.

THE SIGNAL

Poor pruning does not always mean clustering. The question is the shape of the query: a needle, not a range.

admin@sthithapragna.com

> An analytics warehouse is a Medium, correctly sized for its normal workload. Two or three times a week an analyst runs an ad hoc query scanning several terabytes with a selective filter. It takes forty minutes and slows everything else.
>
> The queries are unpredictable, filtering on different columns each time. Finance will not approve permanently running a Large warehouse.

What is the appropriate remedy?

A. Add clustering keys to the large tables so the ad hoc scans prune better.

B. Enable search optimization on the tables, which accelerates large scans.

C. Resize to Large anyway. Ad hoc analysis is a legitimate cost.

D. Enable query acceleration on the warehouse, which leases extra compute only for the eligible queries.

◆ PAUSE AND DECIDE

What is the appropriate remedy?

A Clustering keys on the large tables.

'Filtering on different columns each time' rules it out. A clustering key helps the columns it is defined on, and you cannot cover them all.

B Search optimization for large scans.

SOS is for point lookups and substring, VARIANT or geospatial predicates. It is not a large-scan accelerator.

C Resize to Large permanently.

Pays double for the normal workload all week to help a query that runs three times. Finance has already refused.

D Query acceleration on the warehouse.

Large scans with a selective filter are its eligible shape, and it leases compute only when needed rather than permanently.

THE SIGNAL

'Unpredictable, different columns each time' is the planted detail. It eliminates anything you have to configure in advance.

> A finance dashboard runs the same aggregation over a single 3 TB transactions table roughly two hundred times a day. The result must reflect every transaction the instant it is committed, with no staleness whatsoever.
>
> The account is on Enterprise Edition. The query involves no joins and no window functions.

Which feature fits?

A. A dynamic table with TARGET_LAG = '1 minute', which is the minimum available.

B. Query acceleration, which will speed up the aggregation each time it runs.

C. A materialized view, which is maintained automatically and has no lag.

D. Search optimization on the transactions table, which accelerates aggregate queries.

◆ PAUSE AND DECIDE

Which feature fits?

A A dynamic table at 1 minute lag.

Correct that 1 minute is the minimum, and 'no staleness whatsoever' rules out any lag at all.

B Query acceleration.

QAS accelerates scan and filter work. Aggregation is explicitly not accelerated, and it recomputes every time regardless.

C A materialized view.

One table, no joins, no window functions, repeated aggregation, always current. Every condition is satisfied.

D Search optimization.

Built for point lookups and substring, VARIANT or geospatial predicates. It does not accelerate aggregations.

THE SIGNAL

The scenario states 'no joins and no window functions' deliberately. That sentence exists to confirm a materialized view is legal here.

A team runs ALTER TABLE events ADD SEARCH OPTIMIZATION on a large table. Point lookups on the ordinary id column improve substantially, and the storage bill rises as expected.

However, queries filtering on payload:user.email, a path inside a VARIANT column, are exactly as slow as before. The team believes search optimization is broken.

What is happening?

A Enabling at table level does not cover semi-structured columns. VARIANT needs an explicit ON EQUALITY clause.

B Search optimization does not support VARIANT columns at all. Flatten the payload into typed columns instead.

C The search access path is still building. VARIANT columns are always optimised last.

D The query must cast the VARIANT path explicitly before search optimization will apply.

◆ PAUSE AND DECIDE

What is happening?

A Table-level does not cover semi-structured. Use ON EQUALITY.

Documented explicitly. Queries against VARIANT, OBJECT and ARRAY are not optimised if you omit the ON clause.

B VARIANT is not supported.

It is well supported: equality, IN, ARRAY_CONTAINS, ARRAYS_OVERLAP, SEARCH, substring and NULL checks. It just needs naming.

C Still building, VARIANT last.

Building does take significant time, and there is no documented ordering that leaves VARIANT until the end.

D The path must be cast explicitly.

Casts are supported and sometimes useful, and matching a VARIANT element against a constant works without one.

THE SIGNAL

The other columns improving is the clue. The feature is working; it was simply never switched on for the column that matters.

What to carry into the next module

- **Clustering for ranges, search optimization for needles.** Medium cardinality against high cardinality.
- **Materialized view for zero lag on one table. Dynamic table when a join is involved.**
- **Query acceleration for occasional huge scans** on a warehouse that is otherwise the right size.
- **Table-level search optimization skips semi-structured columns.** VARIANT, OBJECT and ARRAY need an ON clause.
- **Only clustering is Standard Edition.** A scenario naming Standard eliminates the other three.

> ON THE EXAM
>
> Next: SQL features, semi-structured and unstructured data. QUALIFY, PIVOT, MERGE, VARIANT and FLATTEN.